

Mizar Evo **の設計指針について**
自動証明・数学的記述・検証可能な計算の再接続

中正 和久

岩手県立大学

TPP 2026、**理化学研究所 AIP 東京オフィス**、2026 年 11 月 16–17 日

0.3 - 現行 Mizar の課題と 6 つの設計指針

現行 Mizar から取り組む課題	新仕様の設計指針
§ 1 基盤: 論理・MML の継承と処理系の刷新	一階論理・集合論を維持し、kernel を小さく保つ
§ 2 記述: 暗黙の型・登録・演算選択の追跡	数学的な抽象化を継承し、暗黙の選択を明示
§ 3 汎用化: 定義・定理・scheme の個別機構	template で共通化の仕組みを統一
§ 4 計算: 証明と実行可能な手続きの接続	algorithm の契約・不変条件・停止性を検査
§ 5 開発基盤: 依存管理・配布・差分検証・IDE	module・namespace・package・差分ビルド・LSP
§ 6 検査・自動化: 外部探索と検査根拠の接続	ATP を活用し、evidence のインスタンス化と SAT 検査

1.1 - 基盤の継承: 論理は保ち、処理系を再整備

数学的記述の層を継承し、
処理系と開発基盤を再構築する。

- 一階論理と Tarski-Grothendieck 集合論を基盤として維持。
- soft type、mode、attribute、registration、structure、**宣言的証明を継承。**
- **証明探索と検査を分離し、受理を小規模な kernel に集約。詳細は § 6。**

1.2 - 現行 Mizar: 集合論的な関数

関数適用は定義された集合論的關係 **exact MML excerpt**

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

型付き関数は部分関数の上の soft type **exact MML excerpt**

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- 関数是对の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

2.1 - 記述の課題: 可読性と抽象化を継承・拡張

ソース上の関数表現 **sketch** (現行 Mizar と Mizar Evo で有効)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- 新仕様: 数学的な記述を継承・拡張し、型・登録・オーバーロードの暗黙の選択を明示・追跡可能にする。

2.2 - Registration: 自動的な連鎖に名前を付ける

ラベルを持つ自動適用規則 specification example

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;
  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

出発点	自動的に得られる事実	記録する規則
S is empty	S is finite	EmptyImpliesFinite
S is finite	S is countable	FiniteImpliesCountable

- 連鎖的な自動適用を保ち、各登録項目のラベルを必須にして、適用経路を記録する。
- 証明がどの規則に依存するかを説明できる。自動適用のために `by` で明示引用する必要はない。

2.3 - 構造: 格納する field と導出する property

データと標準的な値を分ける **specification example**

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

- **field** は格納するデータで、構成子の引数。property の一意な値は、別の実装で与える。
- **zero** の宣言は型を与える。means 実装は存在・一意性を証明し、equals 実装は値を表す項を与える。

2.3a - 継承関係を後から宣言する

AddLoopStr の宣言後に、親への対応付けを追加 **specification example**

```
definition
  inherit AddLoopStr extends LoopStr where
    field carrier from carrier;
    field add from binop;
    property zero from unit;
  end;
end;
```

- 構造の宣言と継承関係を分離。型を定義した後で Rust の trait を実装する構成に近い。
- from で親の役割をリネーム: binop を add、unit を zero に対応付ける。
- 親ごとに一つの宣言。同じメンバー型なら証明不要、型を狭める場合は coherence を証明する。

2.3b - ダイヤモンド継承と Group の定理の再利用

同じ階層にある二つの経路 sketch

```
AddLoopStr -> LoopStr -> Magma
AddLoopStr -> AddMagma -> Magma
```

Group の定理を環の加法ビューで利用 sketch

```
definition
  let T be type extends Group;
  theorem RightUnit[T]:
    for x being Element of T.carrier holds T.binop(x,T.unit) = x
  proof ... end;
end;
RightUnit[R qua AddLoopStr]  :: gives R.add(x,R.zero) = x
```

- **メンバーの起点と継承経路を追跡し、共有を検査。演算のビューは区別したまま保つ。**
- **環の加法ビューは Group。リネームを通じて定理を再利用する。乗法側に必要なのは monoid の構造。**

3.1 - 現行 MML: 関数の和と結果型

例: 関数の和。「各点で値を足す」という構成を共通化。

結果型の登録 **sketch** (複素数値・実数値の別々の入力ブロック)

```
cluster f1 + f2 -> complex-valued;  
cluster f1 + f2 -> real-valued;
```

VALUED_1 の段階	型の情報
点ごとの和を一度定義	結果は Function
値域の型を絞る	複素数・実数それぞれの PartFunc 再定義
元の定義域全体を回復	それぞれの全域性の登録

- 数学的な演算は既に共通化されている。結果型の精緻化は値の種類ごとに付随する。

3.1a - Template: 共通の本体と具体的な結果型

汎用 functor **sketch** (存在・一意性の証明は省略)

```
definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T.carrier;
  func AddDef: Add[T,I](f,g) -> Function of I,T.carrier means
    for i being Element of I holds it.i = T.add(f.i,g.i);
  synonym f +[T,I] g for Add[T,I](f,g);
end;
```

呼び出し (実数 R・複素数 C)¹

結果型

f + g

Function of I,R.carrier

u + v

Function of I,C.carrier

- **synonym で中置の + を与え、型引数は宣言型から推論する。**

¹明示形: f +[R,I] g、u +[C,I] v。省略は宣言型から一意に推論できる場合。継承経路が曖昧なら qua を明示。

述語パラメータ付き定理としての帰納法 specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: `defpred` に続けて `by NatInduction[P], Base, Step`。
- 関数子パラメータは集合ではなく `schema` レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (`PermProduct[T]`、`qua` によるビュー)、および本資料 Backup 4。

4.1 - 計算の課題: 検証可能な algorithm

読みやすい手続きを Hoare 論理で検査する:

$$\{ \text{requires} \} \text{ algorithm body } \{ \text{ensures} \}$$

- 代入・if/else・while・for・return という馴染みのある擬似コード。契約とループ不変条件で条件を記述する。
- Hoare 論理の規則から一階の証明義務を生成し、正しさと、必要な場合は停止性を検査する。
- アルゴリズム自体の検証と、検証済み手続きによる証明支援に用いる。

4.2 - Algorithm: 契約、証明、計算

契約付きのユークリッドの互除法 specification example

```
terminating algorithm EuclidGcdDef: euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end;
  return x;
end;
```

- **terminating** は **requires** を満たすすべての入力での停止を要求。不変条件と y の厳密な減少を検査する。¹
- 検証後は数学的な functor に昇格し、論理式・証明の中で使える。

¹ループ注釈は Dafny の invariant / decreases に類似。Evo は invariant / decreasing を使う。

4.2a - terminating と functor 昇格

algorithm の形	requires の下での意味
terminating なし	呼出しが戻れば契約が成立
terminating を検証済み	requires の下で全域性。functor として利用

互除法は契約を通じて推論する sketch

```
let a, b be Nat;  
assume a >= 1 & b >= 1;  
thus euclid_gcd(a,b) = Gcd(a,b) by EuclidGcdDef;
```

- 検証済みの定義的な断片は方程式も与える。ループを持つ互除法では全域性と契約の公理のみを用いる。
- `by computation` は別に仕様化。MVM 実行・コード抽出は今後の課題。

4.3 - 互除法: 何を証明するのか

1 回の反復: 旧状態 (x, y) 、 $y > 0$ 。 $r = x \bmod y$ とすると、新状態 $(x', y') = (y, r)$ 。

証明義務	数学的な根拠
不変条件の初期成立	$x=a, y=b$ 。 事前条件から正值性
不変条件の保存	$\text{Gcd}(x, y) = \text{Gcd}(y, r)$ 、 $y \geq 1$ 、 $r \geq 0$
Nat 値の測度の減少	$y' = r$ 、 $0 \leq r < y$
終了時の事後条件	$y=0$ 。 $\text{Gcd}(a, b) = \text{Gcd}(x, 0) = x$

- GCD・剰余の補題をライブラリから用い、定理と同じ枠組みで検査する。SAT 単独が算術を知っているわけではない。
- 最後の $y := r$ を $y := x$ に変えると減少を失う。仕様に基づく説明例。

5.1 - 環境部: 著者は何を取り込むのか

現行 Mizar の環境部、ALGSTR 0 を短縮 **sketch**

```
environ
vocabularies ... STRUCT_0 ...;
notations ... STRUCT_0;
constructors ... STRUCT_0 ...;
registrations ... STRUCT_0;
theorems STRUCT_0;
```

一覧

取り込むもの

vocabularies / notations

記号 / 記法

constructors

構成子

registrations / theorems

自動的な型の事実 / 引用する定理

- 役割別に必要な article を選ぶ。notations • definitions は article の順序にも意味がある。

5.1a - import と依存の追跡

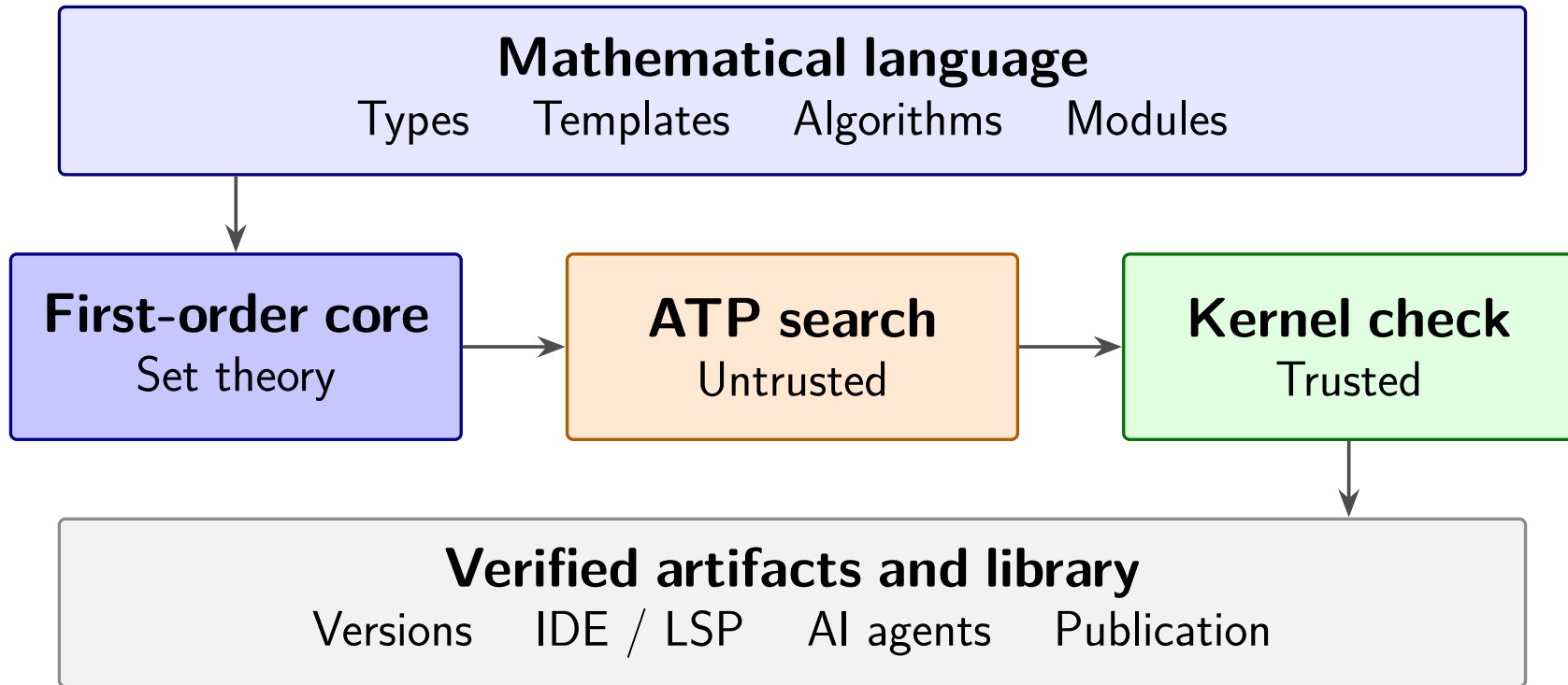
新仕様: モジュールの import specification example

```
import .function;  
import mml.algebra.structure.sorted;
```

段階	著者が確認できるもの
import 解決 再利用	モジュールの公開された定義・定理・登録 元の完全修飾名と、登録の解決トレース 差分ビルドで検証済みの依存指紋

- **公開項目をまとめて取り込み、実際に何を使ったかを追跡する。**
- **package・lock file で依存バージョンを固定し、IDE 診断で解決した名前・トレースを確認する設計。**

5.2 - 全体像



- 豊かな数学的記述を、小規模な一階論理の基盤と現代的な開発環境で支える。

6.1 - 自動化の課題: 証明探索と検査の分離

Isabelle/HOL: Sledgehammer が内部で検証する証明を提案 **sketch**

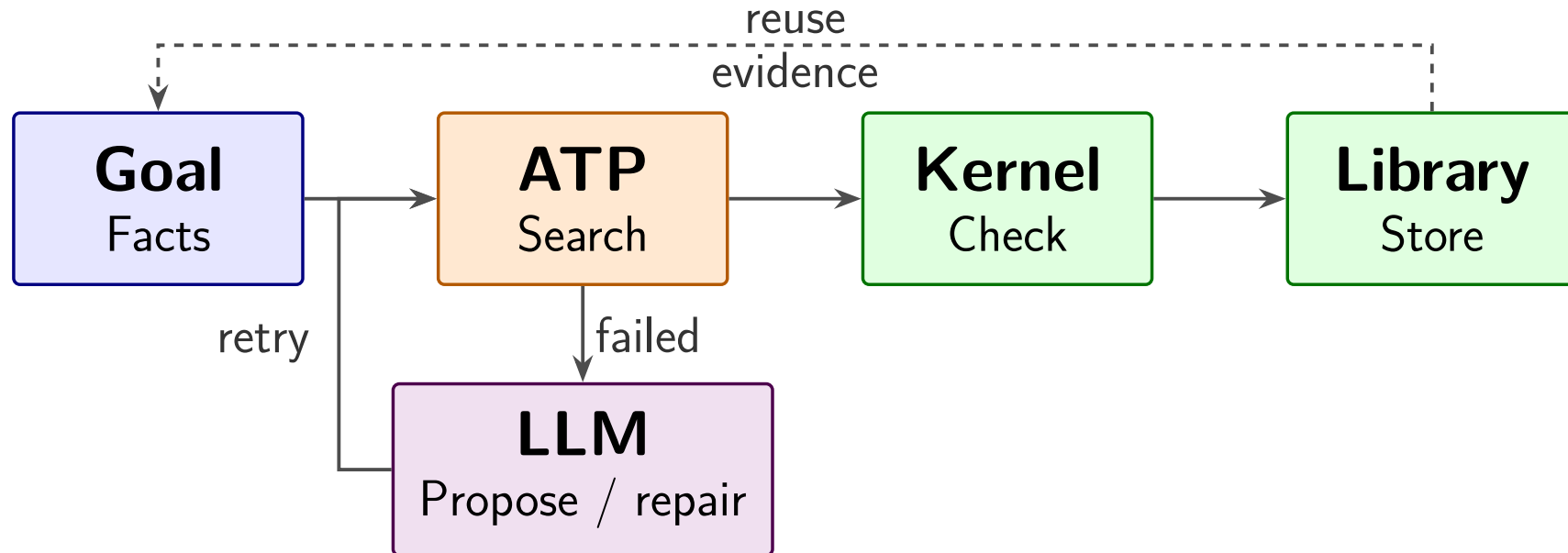
```
have "Q a"  
  sledgehammer  
  by (metis allPQ pa)
```

Mizar Evo: 引用した前提で宣言的な証明ステップを記述 **sketch**

```
assume AllPQ: for x being object holds P(x) implies Q(x);  
assume Pa: P(a);  
thus Q(a) by AllPQ, Pa;
```

- 共通する流れ: ゴールと前提 → 外部探索 → 内側の信頼できる受理判定。prover の成功報告だけでは受理しない。
- Isabelle は Metis 等で内部の証明を再構成。Mizar Evo は論理式・置換の evidence をインスタンス化し、SAT で検査する。

6.2 - ATP の活用: 探索・検査・再利用



- ゴールと前提 → ATP 探索 → kernel 検査 → ライブラリへ保存・再利用。
- 未解決時は補題・方針を修正して再探索。図は設計意図。

6.3 - Resolution 木と置換の抽出

Resolution ログからの候補抽出 sketch

```
F1: forall x. (P(x) implies Q(x)); F2: forall y. (Q(y) implies R(y))
H: P(a); goal: R(a); refute: F1 & F2 & H & not R(a)
```

```
{not P(x), Q(x)}           {not Q(y), R(y)}
  \                         /
  sigma1 = {x := y}         :: unify Q(x), Q(y)
  {not P(y), R(y)}         {P(a)}
    \                       /
    sigma2 = {y := a} :: unify P(y), P(a)
    {R(a)}           {not R(a)}
      \               /
      sigma3 = {}
      {}
```

- 非信頼の抽出器が枝上の置換を合成。F1 に $x := a$ 、F2 に $y := a$ を保存する。

6.3a - 保存する証拠から具体的な SAT 節へ

保存する候補・kernel の前処理・SAT 入力 sketch

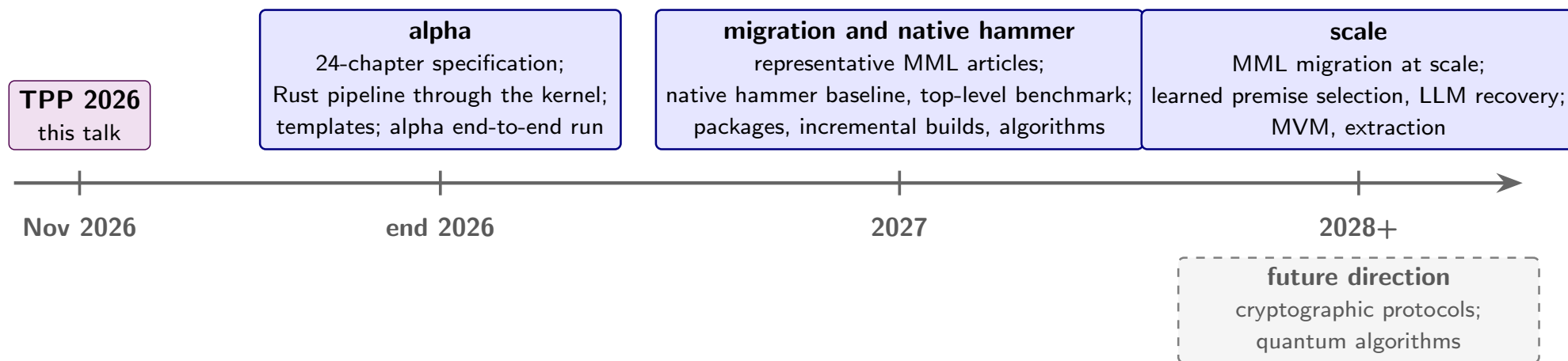
```
save: F1,F2,H (source bindings); F1[x:=a], F2[y:=a]; goal R(a), refute
check: source/context, binders, capture avoidance
instances: P(a) implies Q(a); Q(a) implies R(a); P(a); not R(a)
atoms: p=P(a)=1, q=Q(a)=2, r=R(a)=3
logical CNF: (not p or q) & (not q or r) & p & not r
Tseitin: s=4 iff (not p or q); t=5 iff (not q or r)
SAT clauses (DIMACS example; 0 ends each clause):
p cnf 5 10
1 0          -3 0
1 4 0        -2 4 0      -1 2 -4 0      4 0
2 5 0        -3 5 0      -2 3 -5 0      5 0
```

- small kernel が検査済み evidence から instance・CNF を生成。自身の SAT 検査器で UNSAT を確認し R(a) を受理する。外部 Resolution の各ステップは replay しない。

仕様と実装の状況（2026 年 10 月）

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド（字句解析・構文解析・構文木）、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

ロードマップ



- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行の完成。
- 2027 年: 代表的な MML article の移行、native hammer のベースライン構築、MizAR と同じ top-level theorem 単位での評価。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM・コード抽出。暗号・量子は将来構想。

おわりに

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- 基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。
- 議論したい点: MML の移行、記述の利便性、自動化の評価。

Backup 1. MizAR 60 の詳細

Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

- **データ**: MPTP で出力した MML 1147、**無名の top-level lemma を含む 57,897 件の定理**。MizAR 40 と同一版を用いた比較。
- **学習・開発・holdout を 90:5:5 に分割**。holdout 2,896 件中 1,690 件（58.4%）を hammering mode で証明。**ユーザの前提指定なし、420 CPU 秒**（MizAR 40 は約 40.6%）。
- **人または機械がライブラリから前提を選べる条件では 75% 超**（MizAR 40 は 56%）。
- **最高性能の単一手法**: hammering mode で 30 秒 40%。**人間の前提指定ありで 120 秒 60%**。
- **転移**: 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- **手法**: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

Backup 2. Sledgehammer 評価の詳細

評価	対象・方式	成功率
ITP 2022	AFP 5,000 ゴール、greedy 16 構成	68.8%
Magnushammer (2023 年公開)	PISA 1,000 定理、Sledgehammer	38.3%
同じ PISA 評価	学習による前提選択を用いる Magnushammer	59.5%

- ITP 2022: 50 entry × 100 ゴール。MePo、基準 512 事実、各構成 30 CPU 秒。構成は評価集合から事後選択。再構成は評価対象外。
- PISA: Isabelle2021-1、Sledgehammer の timeout 30 秒、ローカル 5 prover、複数設定の成功集合を集約。Isabelle 内での検証を成功条件とする。
- 年代が近くても、局所ゴールと定理全体、探索成功と検証済み証明を混同しない。

Backup 3. HOL から FOL への符号化と再構成

- 関数適用: 関数変数の適用を `app(F, X)` で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象: `lambda lifting` は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を `guard`、`tag`、または単相化で符号化。選択は健全性・完全性・`prover` 性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による `metis`、SMT 証明の `smt replay`、生成した `Isar` テキスト。再構成の失敗は別途集計。

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

有界な型パラメータと汎用的な定理 **specification example** (証明は省略)

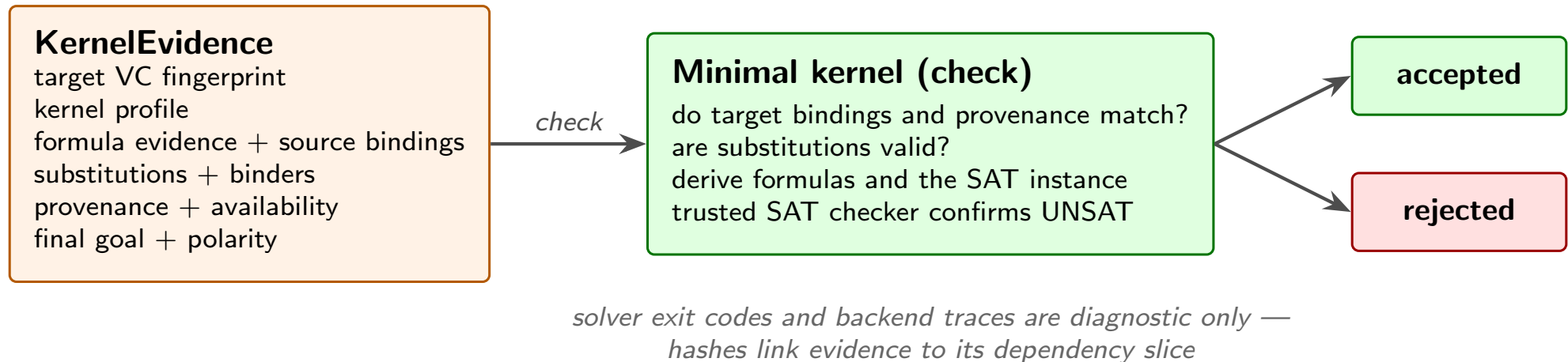
```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

インスタンス化 **specification example** (必要な registration がある前提)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

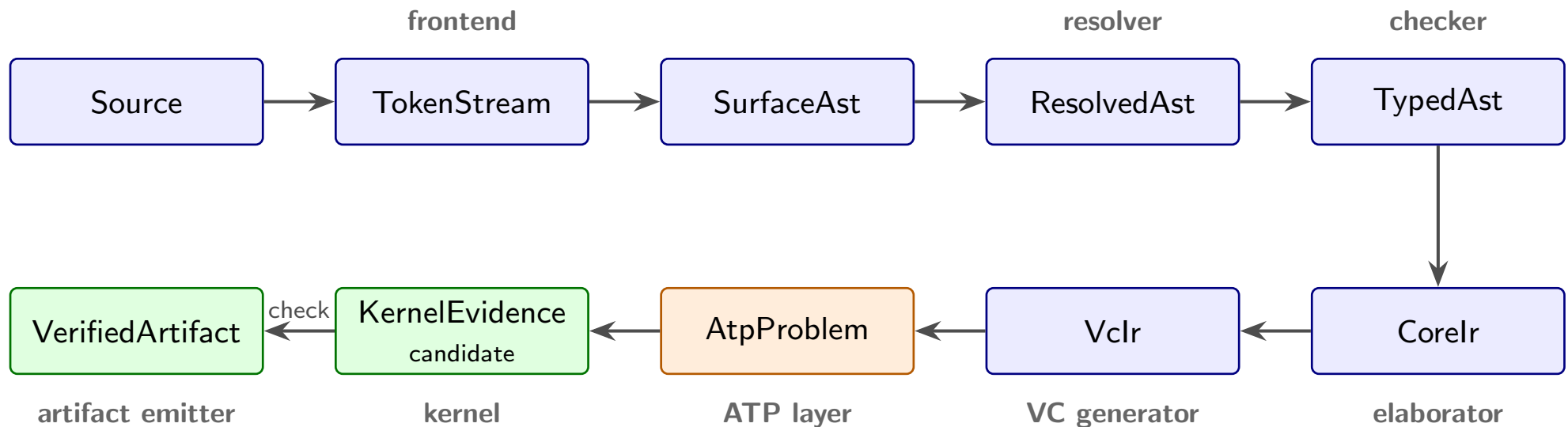
- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

Backup 5. Kernel evidence と信頼できる SAT 検査



- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

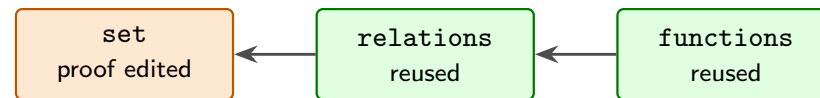
Backup 6. ATP の中心経路



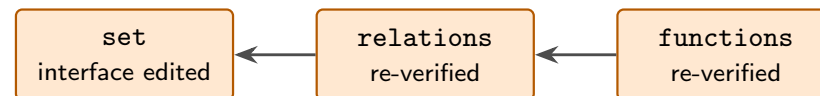
- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

Backup 7. 差分検証

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

Backup 8. Package manifest

Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- **再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。**
- **バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。**

Source: doc/spec/en/23.package_management_and_build_system.md; Białystok 資料 Story 1。

Backup 9. 45 分版での追加

時間に応じ、以下の具体例を追加:

1. 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
2. Mizar の関数表現: Function of X, Y 、 $f.x$ 、集合論的対象としての soft type。
3. Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
4. Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
5. Structure とビュー: AddMagma、MulMagma、Magma (Białystok Story 2)。
6. 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
7. 検査: evidence のインスタンス化と SAT 検査 (本編 6.3、詳細は Backup 5)。

Backup 10. 出典と帰属

本講演における MML の原文引用:

目的	出典	行	使用箇所
関数適用 <code>f.x</code>	<code>funct_1.miz</code>	138-140	Frame 1.2
<code>soft type</code> としての Function of <code>X,Y</code>	<code>funct_2.miz</code>	87-90	Frame 1.2

Source URLs:

- `FUNCT_1`: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- `FUNCT_2`: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

帰属についての注記:

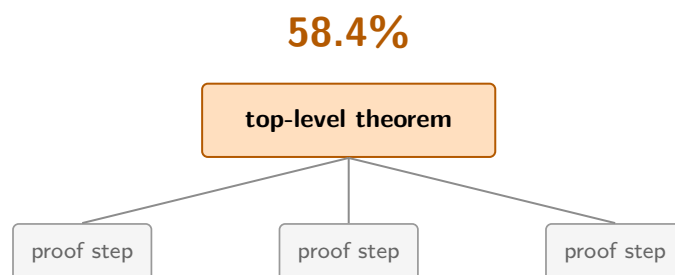
- MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。article 名・URL・行番号を発表者ノートに記載。
- ベンチマークの数値は Backup 1-2 と `references.bib` を参照。最終版の作成前に、出版社の情報で書誌事項を確認。

Backup 11. 評価条件の比較

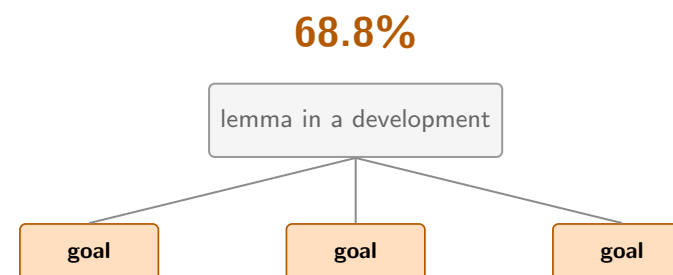
	MizAR 60 (ITP 2023)	Sledgehammer / AFP (ITP 2022)
報告された成功率	58.4% (1,690 / 2,896)	68.8% (3,440 / 5,000)
評価対象	MML 1147 の定理・補題。全 57,897 件のうち holdout 2,896 件	AFP の 50 entry から選んだ局所ゴール 5,000 件
前提選択	ライブラリ全体から学習的に選択	MePo。基準 512 事実、構成により増減
時間予算	ポートフォリオ合計 420 CPU 秒	greedy 16 構成 × 各 30 CPU 秒 = 480 CPU 秒
成功の判定	ATP 証明の発見 (hammering mode)	外部 prover の証明発見。Isabelle 内の再構成は評価対象外

- 評価対象・前提選択・時間予算・構成選択が異なる。成功率だけでは設計上の優劣を判定できない。

Backup 12. 評価単位: 定理全体と局所ゴール



Mizar 60 (ITP 2023) counts *whole*
top-level theorems and lemmas
2,896 holdout problems from 57,897 in
MML 1147; learned premises; 420 CPU s



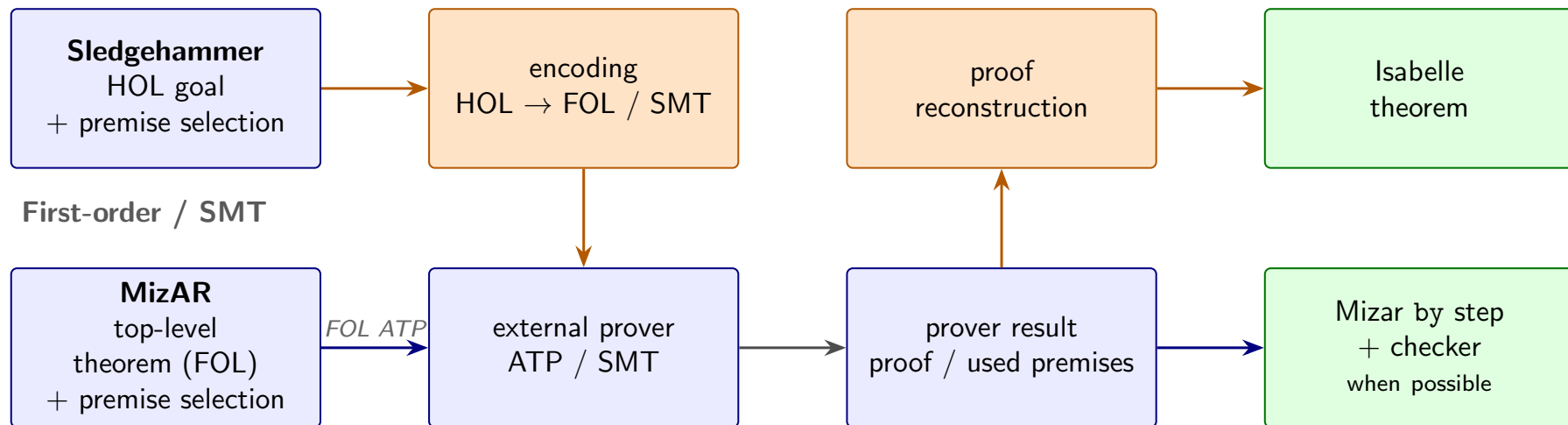
Sledgehammer / AFP (ITP 2022)
counts *proof goals as they arise*
5,000 goals from 50 entries; greedy 16 con-
figurations × 30 CPU s; no reconstruction

Different units, premise selection, and budgets.

- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

Backup 13. 一階 ATP への二つの接続経路

Higher-order (Isabelle/HOL)



Mizar Evo: kernel evidence checking replaces the last step (later slide).

- Sledgehammer の一階・SMT 経路: 高階のゴールを翻訳し、得られた証明を Isabelle 内で再構成する。
- MizAR: 一階・集合論の問題を ATP へ送る。

Backup 14. 高階論理における関数

HOL の文の概形 **sketch** (Isabelle/HOL 風の記法)

```
f :: 'a => 'b      x :: 'a
```

```
P (f x)
```

- HOL: 関数型と関数適用を論理に組み込む。
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- 高階の構造を用いた簡潔な数学的記述が可能。
- E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。

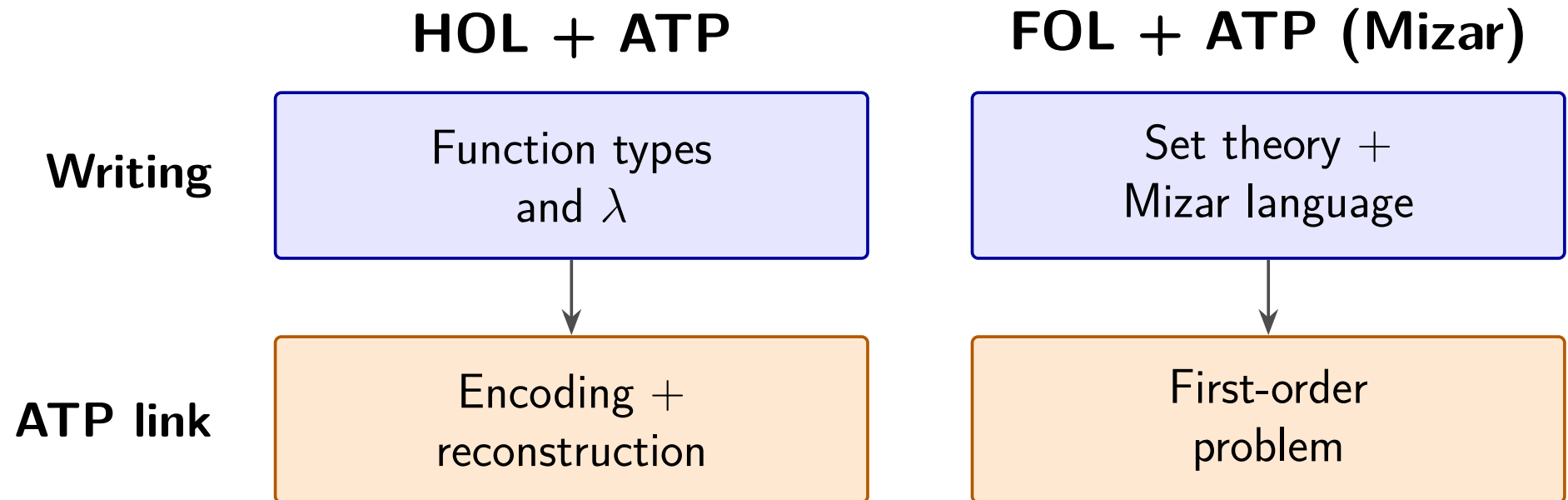
Backup 15. HOL の符号化と再構成

符号化の手順の概形 sketch

$F\ X$	$\rightarrow$	$\text{app}(F, X)$	
$(\%x. t) \dots$	$\rightarrow$	fresh constant + defining axioms	(lambda lifting)
polymorphic types	$\rightarrow$	type guards or type tags	
Boolean-valued terms	$\rightarrow$	extra encoding	

- 関数変数の適用は `app` で明示。ラムダ抽象は `lambda lifting` またはコンビネータで、型は `guard` または `tag` で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : `metis`、`smt` の `replay`、生成した `Isar` テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。
- 接続の仕組みを示す説明であり、実測上の性能の優劣を示すものではない。

Backup 16. 複雑さを担う層の違い



- HOL は高階構造を ATP 向けに符号化。Mizar は集合論の細部を言語層で抽象化。
- Mizar Evo: 一階の基盤を保ち、数学的な記述を現代化。

Backup 17. 設計上のトレードオフ

	HOL ITP + ATP	FOL ITP + ATP
記述	高階機能による簡潔な記述	直接記述すると冗長
関数	基本的な高階の対象	一階の集合論的对象
ATP 接続	高階構造の一階・SMT 符号化	一階問題の生成
再構成	Isabelle 内での再構成	対応可能な Mizar ステップの再検査
言語層の役割	HOL の抽象	一階の細部の抽象化

- 表現と検査の経路を比較した表であり、実測上の性能優位性を示すものではない。